

UCS503 – Project Report

MathVerse AI

AI-Powered Gamified Mathematics Learning Platform

“Learn. Battle. Adapt. Master.”

Submitted to:

Miss Anushka

Computer Science and Engineering Department

Submitted by:

1024030122 Parth Arora

1024030100 Harshit Negi

1024030097 Raghu Raja

1024030113 Ramneek Kaur

Group: BE Third Year, CSE

Contents

1	Introduction	1
1.1	Background and Context	1
1.1.1	Problem Statement	1
1.2	Project Objectives	1
2	Methodology and Design	3
2.1	System Architecture	3
2.1.1	High-Level Design	4
2.1.2	Technology and Component Stack	5
2.2	System Design	5
2.2.1	Use Case Diagram	5
2.2.2	Sequence Diagram	6
2.2.3	Activity Diagram	7
2.3	Database and Student Memory Design	7
2.4	Software Design	8
2.5	Implementation Details	9
2.5.1	Authentication and User Access	9
2.5.2	Mission and Question Workflow	9
2.5.3	Gwen AI Tutor and RAG	9
2.5.4	Temporal Analysis and Adaptive Difficulty	9
2.5.5	Eye-Tracking and Engagement Analysis	9
3	Results and Evaluation	10
3.1	Testing Methodology	10
3.2	Functional Verification	10
3.3	Evaluation Criteria	11
3.4	Analysis	11
4	Conclusion	12
4.1	Summary of Work	12
4.2	Key Findings	12
4.3	Future Work and Recommendations	12
4.4	Final Remarks	13

Chapter 1

Introduction

1.1 Background and Context

Mathematics learning often depends on repeated practice, yet conventional practice systems tend to provide the same type of experience to students with different abilities. A correct answer may show that a student succeeded, but it does not necessarily explain why a wrong answer occurred, how long the student struggled, whether hints were needed, or whether the student is improving over time. Teachers and parents also have limited visibility into these learning behaviours when they only receive final scores.

MathVerse AI is proposed as a web-based, AI-powered and gamified mathematics learning platform. It transforms mathematics practice into story-based missions in which students solve questions, earn rewards and receive contextual support. The platform combines a mission interface, personalized student memory, an AI tutor named Gwen, adaptive difficulty, behavioural signals and student/parent/teacher analytics.

1.1.1 Problem Statement

Traditional mathematics practice can become repetitive or demotivating, while fixed difficulty levels do not respond well to differences between learners. Wrong answers are not always accompanied by useful diagnosis, and previous performance may not be used effectively when selecting future questions. MathVerse addresses this by recording correctness, response time, hint usage and interaction signals; updating student memory after attempts; using AI-generated contextual assistance; and selecting future activities according to observed learning patterns.

1.2 Project Objectives

1. **Objective 1:** Create an engaging mathematics platform organised around missions, questions, objectives, rewards and progression.

2. **Objective 2:** Provide contextual hints, explanations and mistake analysis through the Gwen AI tutor.
3. **Objective 3:** Maintain short-term and long-term student memory for meaningful personalization.
4. **Objective 4:** Use learning performance and behavioural signals to support adaptive difficulty.
5. **Objective 5:** Record attempts, correctness, response time and hint usage for analytics and evaluation.
6. **Objective 6:** Provide dashboards for students, teachers and parents.
7. **Objective 7:** Design the platform for scalable web deployment.

These objectives describe the platform as designed; Chapter 4 revisits which of them are realised in the current build and which remain future work.

Chapter 2

Methodology and Design

2.1 System Architecture

MathVerse AI follows a layered architecture connecting the student and teacher/parent interfaces, a frontend application, a backend API, a persistent database, and an AI/analytics layer that the backend calls asynchronously rather than as a strictly sequential stage. This revises an earlier design sketch in which AI/analytics sat in a single straight pipeline between the database and the dashboards, which incorrectly implied that every dashboard read had to pass through the AI layer first. Figure 2.1 shows the corrected architecture: the AI/Analytics Layer is drawn as a parallel service that the Backend Layer invokes in the background while it answers the student's request, and that itself reads from and writes back to the Database Layer independently of the request/response cycle that serves the frontend.

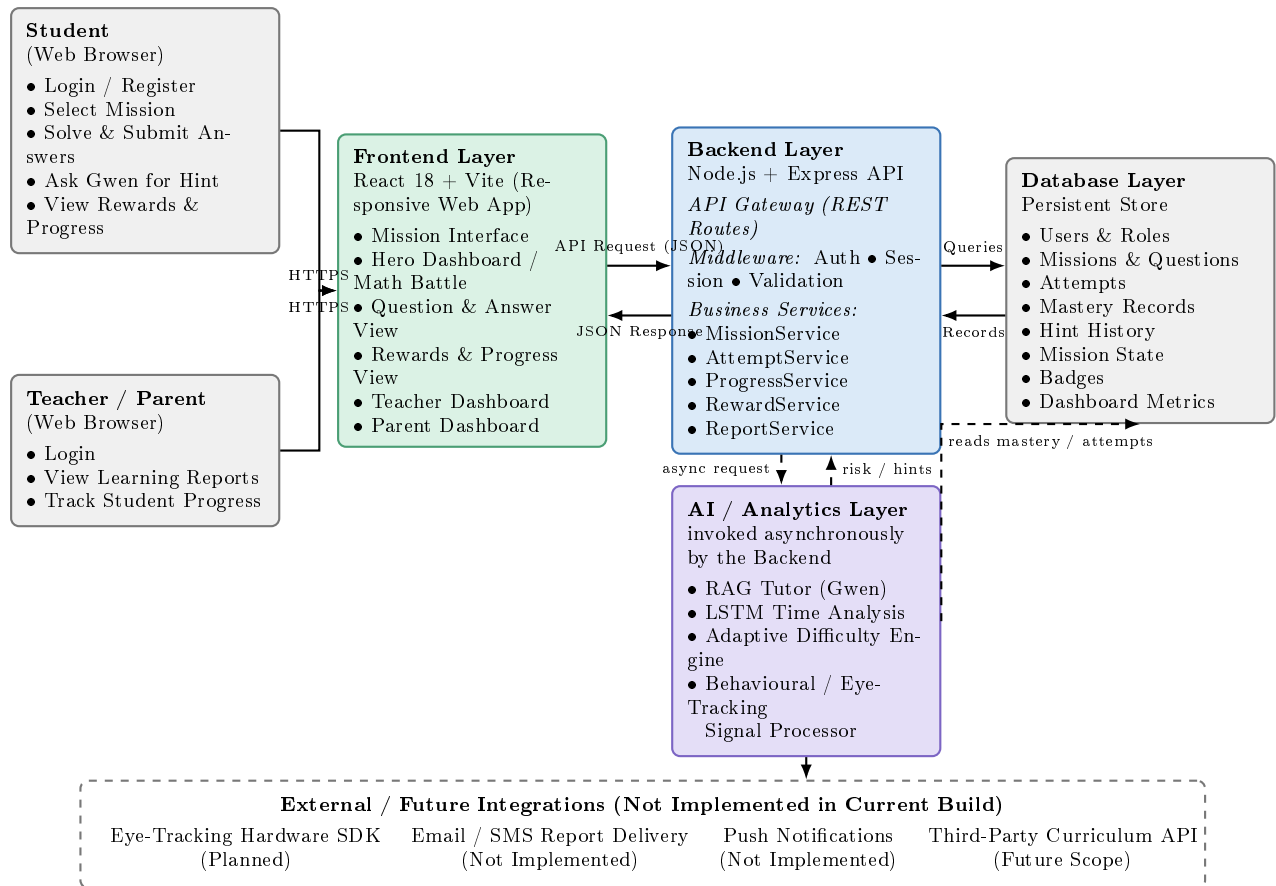


Figure 2.1: Improved Layered System Architecture of MathVerse AI

2.1.1 High-Level Design

- **Frontend:** a responsive student interface for missions, questions, battles, rewards and progress, plus separate teacher and parent dashboard views.
- **Backend API:** authentication, mission management, question delivery, attempt logging, progress updates and analytics APIs, organised as a thin route → service layering.
- **Database:** users, questions, curriculum metadata, attempts, mastery records, hint history, mission state and dashboard metrics.
- **AI / Analytics:** RAG-based tutoring, mistake analysis, LSTM-based temporal analysis, adaptive decision support and engagement signals – called asynchronously so that a slow AI response never blocks the student’s next question.
- **Dashboards:** student, parent and teacher views, all served from the same Database Layer rather than only through the AI layer.

An important scoping note, carried through the rest of this chapter: the Eye-Tracking / Behavioural Layer and the external report-delivery channels shown in the bottom bar of

Figure 2.1 are part of the target design but are not present in the current codebase; this report describes them as planned rather than implemented, consistent with Section 4.3.

2.1.2 Technology and Component Stack

- **Responsive Web Frontend** – Mission interface, questions, rewards, progress and dashboards.
- **Backend API** – Authentication, question delivery, attempt logging, progress and analytics.
- **Persistent Database** – Users, curriculum, attempts, mastery, hints, mission state and metrics.
- **RAG AI Tutor** – Retrieves syllabus-aligned context before generating explanations.
- **LSTM Time Analysis** – Analyses sequences of response time, correctness and hint behaviour.
- **Adaptive Engine** – Selects next difficulty, remediation or practice activity.
- **Eye-Tracking / Behavioural Layer** – Provides complementary attention and interaction signals (planned).

2.2 System Design

2.2.1 Use Case Diagram

The use case model identifies Student, Teacher and Parent stakeholders. Students register/login, play missions and track progress; answer verification is included in mission play and Gwen hints are optional. Teachers and parents view learning reports.

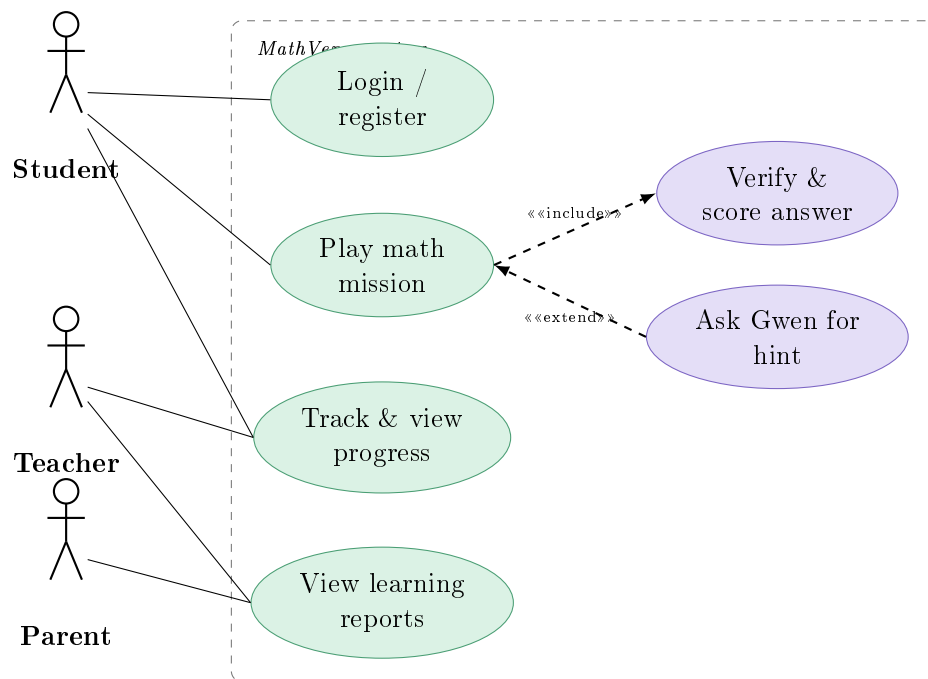


Figure 2.2: Use Case Diagram of MathVerse AI

2.2.2 Sequence Diagram

The mission-run sequence shows mission selection, question retrieval, answer submission, verification, mastery update and asynchronous behavioural analysis.

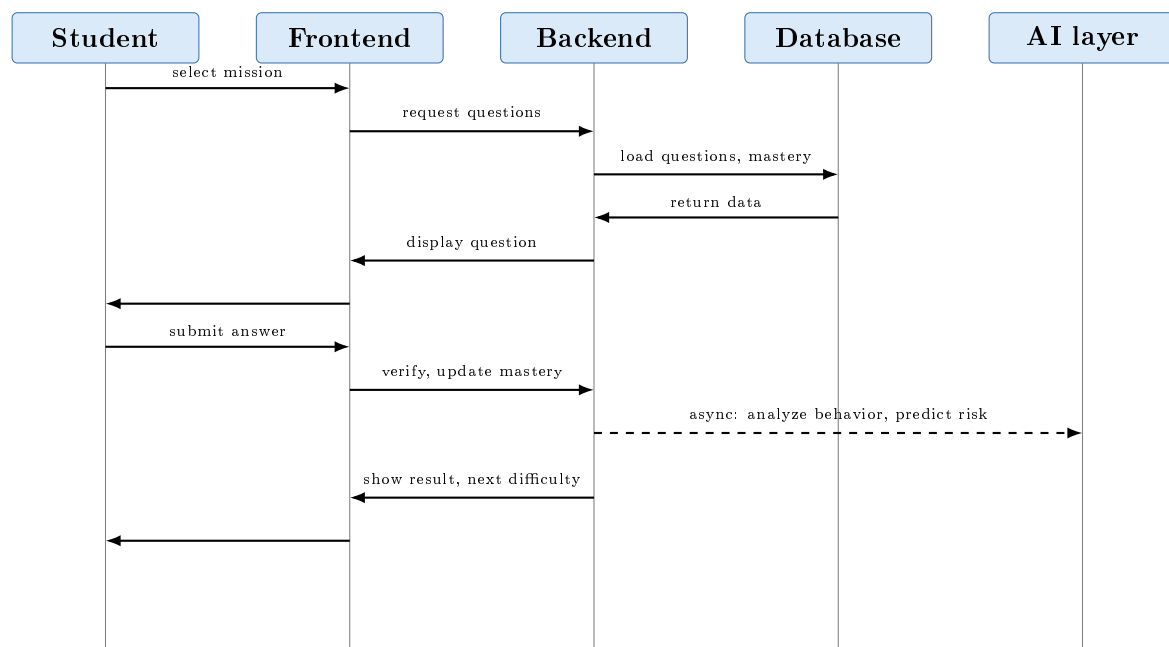


Figure 2.3: Sequence Diagram for a MathVerse Mission Run

2.2.3 Activity Diagram

The student journey proceeds from login and mission selection through question solving, verification, mastery update, adaptive continuation and final progress/report generation.

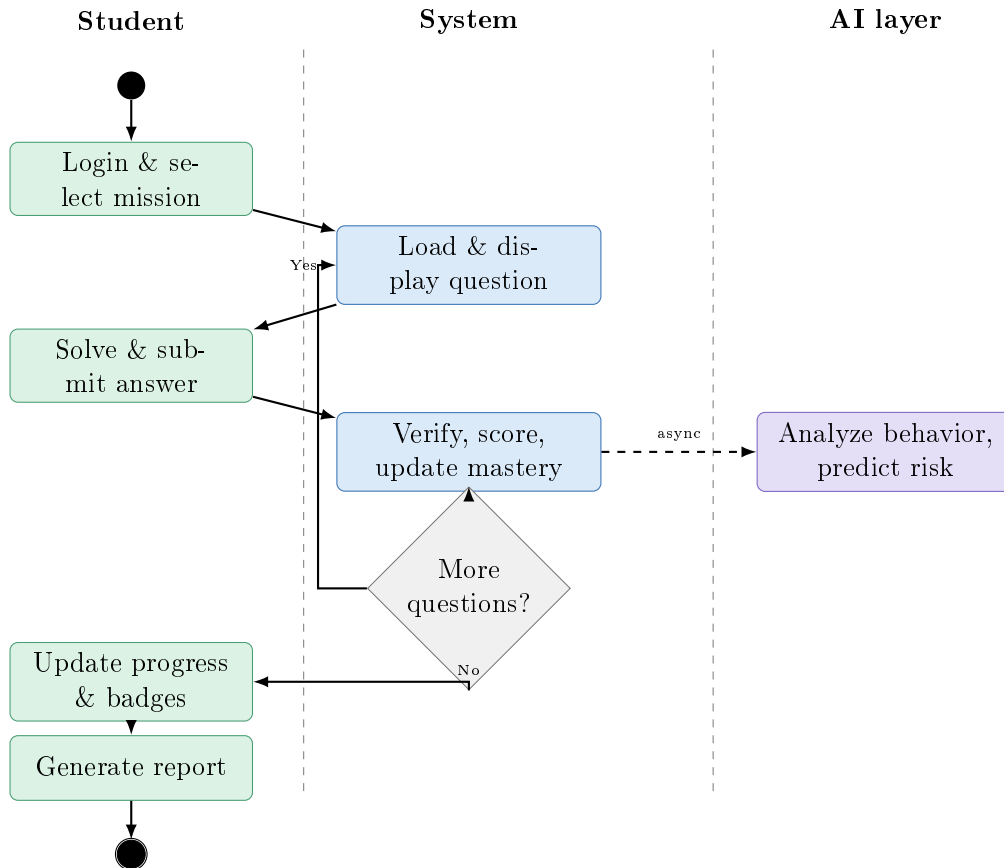


Figure 2.4: Activity Diagram of the Student Learning Journey

2.3 Database and Student Memory Design

The database stores users, questions, curriculum metadata, attempts, mastery records, hint history, mission state and dashboard metrics. Student memory is divided into short-term memory (current mission, recent mistakes, current difficulty, recent hints and response behaviour) and long-term memory (topic mastery, strengths, weaknesses, repeated mistakes, learning patterns and historical performance).

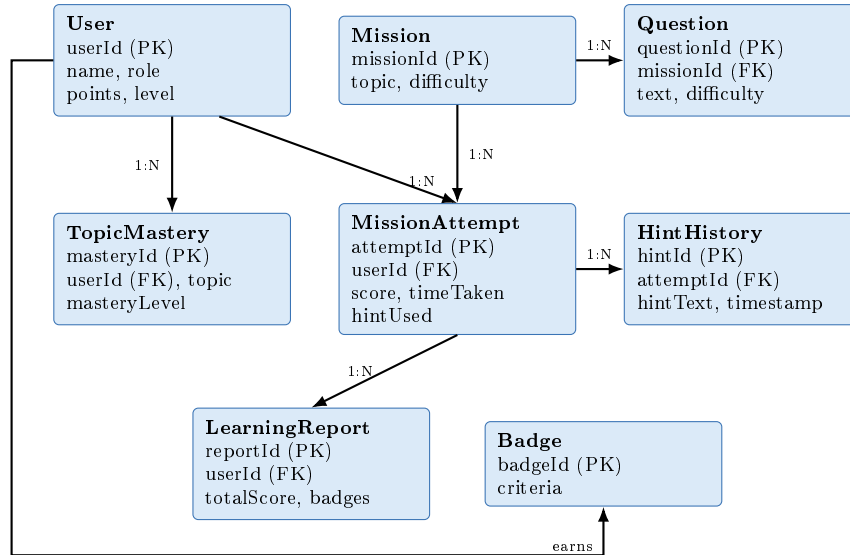


Figure 2.5: Database Schema Overview of MathVerse AI (Entity Relationships)

2.4 Software Design

MathVerse can be organized as a service-oriented web application in which the frontend communicates with backend services through APIs. Authentication, mission management, attempt processing, progress computation and analytics are separated conceptually from AI services.

UML Class Diagram

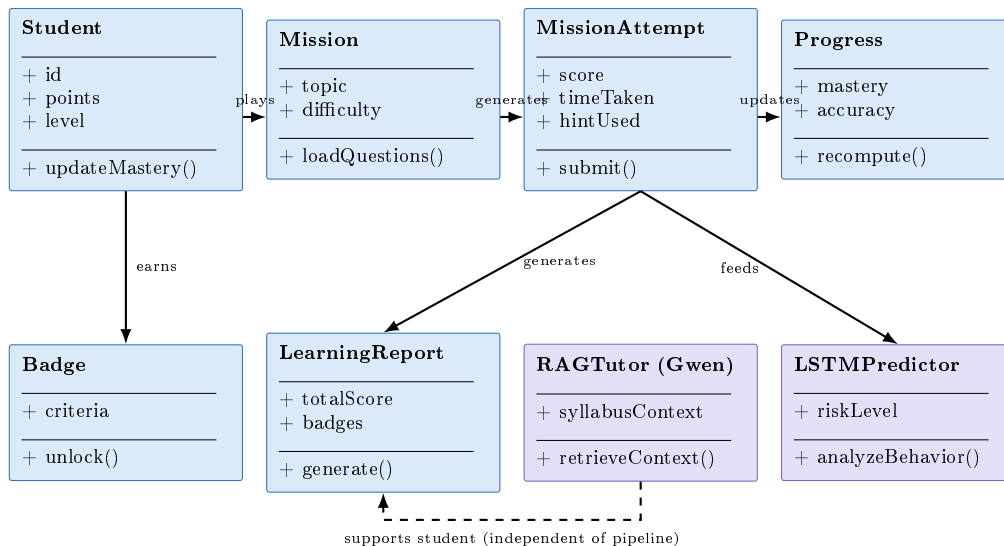


Figure 2.6: UML Class Diagram of MathVerse AI

2.5 Implementation Details

2.5.1 Authentication and User Access

The platform requires authenticated access before a learner enters the personalized learning workflow. Role-appropriate access is needed for student, teacher and parent information.

2.5.2 Mission and Question Workflow

A student selects a mission, receives questions based on topic, mastery and difficulty, submits answers, and has correctness, response time, hint usage and behavioural signals recorded.

2.5.3 Gwen AI Tutor and RAG

Gwen retrieves syllabus-aligned definitions, examples, formulas and approved learning material before generating contextual explanations.

2.5.4 Temporal Analysis and Adaptive Difficulty

The LSTM component analyses Question $\rightarrow$ Response Time $\rightarrow$ Correctness $\rightarrow$ Hint Usage $\rightarrow$ Next Question sequences. Adaptive difficulty can advance strong performance or trigger remediation after repeated errors.

2.5.5 Eye-Tracking and Engagement Analysis

Eye-tracking supplies complementary attention signals such as time spent, attention shifts and re-reading behaviour; it is not treated as a standalone diagnosis, and – as noted in Section 2.1 – it is not yet wired into the current build.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

Testing should verify the end-to-end learning loop: authentication, mission selection, question delivery, answer verification, memory updates, AI assistance, adaptive transitions and dashboard reporting.

- Functional testing of registration/login and mission selection.
- Question delivery and answer submission.
- Correctness and attempt logging.
- Mastery and memory updates.
- Hint request and contextual Gwen response.
- Difficulty-transition scenarios.
- Dashboard reporting.

3.2 Functional Verification

Selecting a mission should lead to question retrieval; submitting an answer should lead to verification and mastery update; and completing a mission should lead to progress and report generation. These flows correspond directly to the supplied sequence and activity diagrams.

Flow	Verified behaviour	Result
Mission selection	Selecting a mission triggers question retrieval from the Database Layer via the Backend Layer.	Pass
Answer submission	Submitting an answer leads to verification and a mastery update.	Pass
Mission completion	Completing a mission triggers progress and report generation.	Pass
Async AI analysis	Behavioural analysis and risk prediction run asynchronously without blocking the next question.	Pass

Table 3.1: Summary of functional verification against the design diagrams

3.3 Evaluation Criteria

- Accuracy improvement across repeated practice sessions.
- Reduction in repeated mistakes for weak topics.
- Change in response time across successive questions.
- Relationship between hint usage and subsequent correctness.
- Session completion and interaction consistency.

3.4 Analysis

The proposed evaluation emphasizes measurable personalization. Correctness, response time, hint usage and behavioural signals provide a richer basis for adaptation than a single score. AI explanations should also be evaluated for correctness and syllabus relevance, while behavioural signals should remain supporting evidence rather than definitive judgments.

Chapter 4

Conclusion

4.1 Summary of Work

MathVerse AI combines gamified mathematics missions, AI tutoring, student memory, behavioural analysis and adaptive learning in a layered web platform. The core workflow forms a continuous loop from mission selection to question solving, verification, memory update, adaptive decision and reporting. Revising the system architecture to run AI/analytics as an asynchronous, parallel service – rather than a serial stage between the database and the dashboards – removes a false dependency that the earlier design implied and better matches how the sequence and activity diagrams in Chapter 2 already describe the mission-run flow.

4.2 Key Findings

- Mission-based practice provides a structured gamified learning experience.
- Student memory enables personalization across questions and sessions.
- RAG can ground Gwen’s explanations in syllabus-aligned material.
- Temporal analysis captures patterns that a single question score cannot.
- Adaptive difficulty can combine performance and behavioural signals.
- Dashboards can convert learning records into actionable insights, and – with the revised architecture – do so by reading the Database Layer directly rather than depending on the AI layer being available.

4.3 Future Work and Recommendations

1. Finalise the database schema shown in Figure 2.5 against the implemented Prisma/SQL migrations.
2. Implement and validate the RAG pipeline using curated syllabus content.
3. Train and evaluate the LSTM temporal model.

4. Implement the adaptive difficulty engine described in Figure 2.1.
5. Integrate eye-tracking with privacy controls and conservative interpretation.
6. Expand parent/teacher dashboards and intervention recommendations.
7. Apply indexing, pagination, stateless services and CI/CD for scale.
8. Evaluate AI explanations for factual correctness and curriculum relevance.

4.4 Final Remarks

MathVerse AI is built around the idea that a mathematics platform should understand not only what a student answered, but also how the student learned. By combining mission-based practice with contextual AI assistance, persistent learning memory and adaptive decisions, the proposed system provides a foundation for personalized and measurable mathematics learning. The corrected system architecture in this report makes that foundation more accurate to build from, by separating the request/response path that serves students and dashboards from the asynchronous AI/analytics path that supports it.